Error Log

Noorinder

1. Missing Validation for Withdrawals

- **Error:** The `withdraw()` method allowed negative amounts, causing the balance to increase instead of decrease.
- **Cause:** There was no validation for positive withdrawal amounts.
- **Fix:** Added a check to ensure withdrawal amounts were greater than zero.

Before Fix:

```
public void withdraw(double amount) {
    balance -= amount; // No validation
}
```

After Fix:

```
  public void withdraw(double amount) {
      if (amount <= 0) {
          throw new IllegalArgumentException("Withdrawal amount must be positive.");
      }
      balance -= amount;
  }
```

2. Incorrect Penalty Application

- **Error:** Penalties were applied even when the withdrawal couldn't be completed due to insufficient funds.
- **Cause:** The penalty logic didn't account for whether the withdrawal succeeded.
- **Fix:** Adjusted the logic to apply penalties only after successful withdrawals.

Before Fix:

```
if (balance - amount < 100) {
    balance -= 2; // Penalty applied before withdrawal check
}
if (balance - amount >= 0) {
    balance -= amount;
}
```

After Fix:

```
if (balance - amount >= 0) {
    if (balance - amount < 100) {
        balance -= 2; // Apply penalty after checking sufficient funds
    }
    balance -= amount;
} else {
    throw new IllegalArgumentException("Insufficient funds for withdrawal.");
}
```

3. Invalid Account Type Handling

- **Error:** Entering an invalid account type caused crashes or undefined behavior.
- **Cause:** The `switch` statement didn't handle invalid inputs.
- **Fix:** Added a `default` case to display an error message and re-prompt the user.

Before Fix:

```
    switch (acctType) {
    case 1:
        account = personalAcct;
        break;
    case 2:
        account = businessAcct;
        break;
    // No handling for invalid inputs
}
```

After Fix:

```
    switch (acctType) {
    case 1:
        account = personalAcct;
        break;
    case 2:
        account = businessAcct;
        break;
    default:
        System.out.println("Invalid Account type! Please try again.");
        continue; // Re-prompt the user
}
```

4. Negative Initial Balances

- **Error:** The program allowed negative balances when creating accounts, leading to inconsistent behavior.
- **Cause:** The `Account` constructor didn't validate the initial balance.
- **Fix:** Added a validation check to ensure the balance is non-negative.

Before Fix:

```
public Account(double balance) {
    this.balance = balance; // No validation
}
```

After Fix:

```
public Account(double balance) {
    if (balance < 0) {
        throw new IllegalArgumentException("Initial balance cannot be negative.");
    }
    this.balance = balance;
}
```

5. No Quit Confirmation

- **Error:** The program exited immediately when the quit option was selected, causing accidental terminations.
- **Cause:** There was no confirmation prompt for quitting.
- **Fix:** Added a confirmation prompt to verify the user's intent before exiting.

Before Fix:

```
if (action.equalsIgnoreCase("Q")) {
    break; // Immediate exit
}
```

After Fix:

```java
if (action.equalsIgnoreCase("Q")) {
    System.out.print("Are you sure you want to quit? (yes/no): ");
    String confirmation = input.next();
    if (!confirmation.equalsIgnoreCase("yes")) {
        continue; // Do not exit if the user changes their mind
    }
}
```